arXiv:2602.09311v2 [cs.SE] 28 May 2026

Cross-Project Flakiness: A Case Study of the OpenStack Ecosystem

Tao Xiao, Dong Wang ✉, Shane McIntosh, Hideaki Hata, Yasutaka Kamei

***Abstract*—Automated regression testing is a cornerstone of modern software development, often contributing directly to code review and Continuous Integration (CI). Yet some tests suffer from flakiness, where their outcomes vary non-deterministically. Flakiness erodes developer trust in test results, wastes computational resources, and undermines CI reliability. While prior research has examined test flakiness within individual projects, its broader ecosystem-wide impact remains largely unexplored. In this paper, we present an empirical study of test flakiness in the OpenStack ecosystem, which focuses on (1) cross-project flakiness, where flaky tests impact multiple projects, and (2) inconsistent flakiness, where a test exhibits flakiness in some projects but remains stable in others. By analyzing 649 OpenStack projects, we identify 1,535 cross-project flaky tests and 1,105 inconsistently flaky tests. We find that cross-project flakiness affects 55% of OpenStack projects and significantly increases both review time and computational costs. Surprisingly, 70% of unit tests exhibit cross-project flakiness, challenging the assumption that unit tests are inherently insulated from issues that span modules like integration and system-level tests. Through qualitative analysis, we observe that race conditions in CI, inconsistent build configurations, and dependency mismatches are the primary causes of inconsistent flakiness. These findings underline the need for better coordination across complex ecosystems, standardized CI configurations, and improved test isolation strategies.**

***Index Terms*—Continuous Integration, Flaky Test, OpenStack**

I. Introduction

DEVELOPERS rely on automated regression testing to assess whether key functionality is perturbed by their changes. Ideally, each test failure should serve as a clear indication of problems in a change set; however, failures may occur due to non-deterministic factors. This behavior, known as **test flakiness** [33], provides a misleading signal to developers regarding their changes. Flakiness arises when a test produces both passing and failing results when re-executed under identical conditions. Prior research has highlighted issues caused by test flakiness, such as wasted developer effort in investigating false positives [69, 78], reduced efficiency of Continuous Integration (CI) systems [14, 29], and the obfuscation of software defects [70, 78].

Addressing test flakiness is a persistent challenge in software testing. For example, Google has documented that between 2% and 16% of its computational resources are allocated to re-execute flaky tests [40]. A survey of 58 Microsoft developers found that they ranked test flakiness as the second most important perceived barrier to software deployment [28]. The economic impact of test flakiness is also substantial. Herzig et al. [21] estimated that the cost of investigating test failures caused by flakiness at Microsoft amounts to approximately $7.2 million for a single product. Additionally, an analysis of GitHub commits revealed that "1 in 11 commits had at least one [failure] caused by a flaky test" [61]. Similar issues have been reported by Mozilla [7] and Huawei [24], further indicating the widespread impact of test flakiness.

Prior research has predominantly focused on flakiness at the programming level [59]. To the best of our knowledge, the cascading impact of flakiness at the ecosystem level, which encompasses a set of socio-technically interdependent software projects [23, 35, 38], has not yet been explored. While individual projects within the ecosystem are managed by their respective project teams or organizations, the strength of the ecosystem stems from the social interactions among these teams (both within and across projects [12, 39]) as well as their ability to foster reuse functionalities through technical dependencies [1, 6, 13, 26]. Flakiness in such an ecosystem presents unique challenges since it does not remain insulated within a single project. Instead, it propagates through shared dependencies and interfaces that multiple projects rely on.

In this paper, we perform a case study of OpenStack, a renowned infrastructure-based ecosystem [19] developed by 50 participating industrial members, including large-scale and well-known companies, such as AMD, Microsoft, and Cisco. Through our preliminary exploration, we identify **cross-project flakiness** and **inconsistent flakiness** phenomena in the OpenStack ecosystem (detailed in Section III). Cross-project flakiness occurs when a flaky test affects multiple projects. Inconsistent flakiness refers to instances where a test exhibits non-deterministic behavior in a subset of projects while remaining stable in others. While prior research [33, 34] has explored test flakiness within individual projects, its characteristics and impact on multiple projects, particularly at the ecosystem level, remain largely unexplored. More specifically, we address the following three Research Questions (RQs):

RQ1. To what extent does cross-project flakiness exist?

Motivation. The prevalence of cross-project flakiness in the OpenStack ecosystem remains unclear, highlighting a critical knowledge gap given the limited resources available to address the issue. Therefore, we first set out to quantify the prevalence of cross-project flakiness. Providing a foundational understanding of its behavior for assisting team leads of software

T. Xiao, Y. Kamei are with Kyushu University, Japan.
E-mail: {xiao, kamei}@ait.kyushu-u.ac.jp
D. Wang is with Tianjin University, China.
E-mail: dong_w@tju.edu.cn
S. McIntosh is with the University of Waterloo, Canada.
E-mail: shane.mcintosh@uwaterloo.ca
H. Hata is with Shinshu University, Japan.
E-mail: hata@shinshu-u.ac.jp

ecosystems, responsible for overseeing all technical aspects and representing contributors, in making informed decisions for effective intervention.

Results. We find that cross-project flakiness impacts a large proportion (55%) of OpenStack projects, occurring with greater frequency than flakiness confined to individual projects. Moreover, cross-project flakiness has rapidly accumulated over time in OpenStack. These findings are a call to action for addressing cross-project flakiness to enhance CI reliability and reduce resource consumption.

RQ2. How do test scopes influence the likelihood of cross-project flakiness?

Motivations. Software testing can be categorized into various types to comprehensively validate the software, such as unit tests, functional tests, and integration tests. Each test type may interact with system dependencies and infrastructure in unique ways, which could potentially lead to different degrees of flakiness (e.g., number of projects being affected by the flaky test) in interconnected ecosystems like OpenStack. By exploring how these various test types contribute to cross-project flakiness, researchers and developers can pinpoint which test types are most vulnerable to inducing such instability. Consequently, team leads of software ecosystems could leverage these insights to refine existing test design strategies and proactively address identified flakiness phenomena.

Results. We find that the two flakiness phenomena are prevalent in the OpenStack ecosystem, with 1,535 and 1,105 flaky tests presenting with cross-project flakiness and inconsistent flakiness, respectively. Unit tests are the most likely to exhibit flakiness, accounting for 7,202 flaky tests observed across 30,550 test executions. Moreover, we observe a strong correlation between test scopes (i.e., unit, integration, system, scenario, or API tests) and flakiness levels (i.e., the extent to which flakiness propagates across the network of projects). We find that 64% of API tests and 41% of scenario tests are prone to causing flakiness across multiple OpenStack projects. Surprisingly, cross-project flakiness affects 70% of unit tests, suggesting that even tests that are traditionally well-isolated can contribute to ecosystem-wide flakiness. These findings suggest that test strategies may benefit from refinement, especially in terms of test isolation and resource management, to mitigate flakiness at its root cause.

RQ3. What are the causes of inconsistent flakiness?

Motivations. In this RQ, we conduct an in-depth analysis to uncover the underlying causes of inconsistent flakiness, that is, why certain projects experience flakiness under similar conditions while others do not. Understanding these causes not only helps in developing targeted mitigation strategies but also guides the refinement of testing practices and tools for interconnected ecosystems, ultimately leading to more stable and robust software systems benefiting all stakeholders.

Results. We find that race conditions in the CI process are the predominant cause of inconsistent flakiness, accounting for 89% of cases. Additionally, both mismatched CI configurations and dependency management issues contribute to 21% of cases. These observations call for standardized CI configurations and enhanced coordination among OpenStack projects to reduce inconsistencies.

To understand how OpenStack developers perceive our empirical observations, we solicit their feedback through a questionnaire. We received valid responses from 15 contributors, core developers, and project maintainers. The respondents explain that OpenStack stakeholders have also encountered the two flakiness phenomena. Moreover, the respondents commented on and proposed mitigation strategies, such as the shift from the "recheck and wait" paradigm to early intervention on flaky tests or standardizing CI configuration and environment.

To facilitate replication and promote future work, we have made a replication package publicly available [75], which includes our raw data set, manually labelled data, anonymized questionnaire responses, and analysis code scripts.

II. Code Review and Continuous Integration

To investigate test flakiness at the ecosystem level, we explain its occurrence in practice, particularly in OpenStack. Modern code review is a lightweight process widely adopted in both industrial and open-source settings [64, 72], conducted online and asynchronously through review tools such as Gerrit.[1] The process typically commences when a code author submits a set of changes to a code base, along with a description of the changes, to the review tool. Then, reviewers critique the premise, content, and structure of the submitted changes, providing feedback for the author to refine their work.

While authors may test their changes locally before submission, not all developers have the necessary resources to run the complete test suite. Consequently, reviewers cannot assume that all relevant tests have been conducted. To address this limitation, the code review process is tightly integrated with CI through bots (e.g., the Zuul bot of OpenStack) that automatically perform builds and run regression tests. Along with reviewers' feedback, CI bots report the results of builds and tests. Authors may upload revised change sets in response to reviewer feedback and to satisfy CI bots. This cycle of review, testing, and revision continues until the latest revision meets both the CI bots' criteria and the reviewers' expectations. Throughout this paper, in accordance with `Gerrit` terminology, a *change set* is referred to as a *code review*, and each *revision of a change set* is referred to as a *patch set*.

III. Real-World Example of Cross-Project Flakiness

Figure 1 presents a real-world example of cross-project flakiness observed during code review `#877934`[2] in Cinder and `#882133`[3] in Glance from the OpenStack ecosystem. In this example, (A) denotes the patch set on which the Zuul build was based, while (B) represents the build status and the triggering pipelines. (C) refers to the build jobs executed by Zuul, while (D) represents the `recheck` (a re-run technique to mitigate non-deterministic behavior) request invoked by the author. (E) indicates the review comment explaining the

[1] https://www.gerritcodereview.com/
[2] https://review.opendev.org/c/openstack/cinder/+/877934
[3] https://review.opendev.org/c/openstack/glance/+/882133

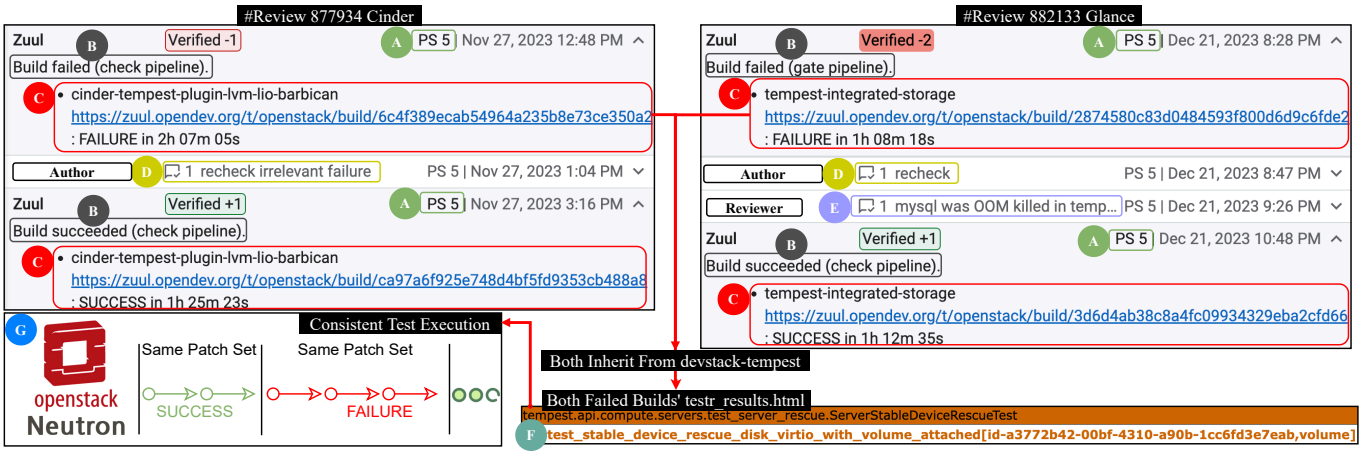


Fig. 1: Real-world example of cross-project flakiness during the code review #877934 (Cinder project) and #882133 (Glance project) from OpenStack community.

random failure provided by the patch reviewer, while both failing builds pinpoint to the same failing test (F). Finally, this test can also be executed in other projects, shown as G.

OpenStack's Zuul CI workflow is event-driven, where new changes trigger different pipelines based on their review status, such as *check* pipeline (when uploading a new patch set) or *gate* pipeline (before merging changes). Once the *gate* has failed, the submitted patch set must proceed through a *check* pipeline similar to B (transitioning from *gate* to *check*) on the right side of Figure 1. In OpenStack, build jobs and test suites could be shared across different projects through inheritance from common job definitions. For example, both C jobs from two reviews inherit from common parents, `devstack-tempest` in the Tempest project. These two jobs in Cinder and Glance executed the same set of standard Tempest API tests to ensure that a modification in one project does not break the OpenStack API.

As shown in the figure, the authors of both changes request a rerun of the CI job after its failure by initiating a `recheck` command (D). The two different statuses of the two CI builds, "SUCCESS" and "FAILURE," are shown in C. These statuses are from the Zuul CI, and both jobs are based on patch set 5 (A). In C, we refer to the failing CI builds as the **flaky build**, the unique identifier in the URL as the identifier; and the failing CI jobs as **flaky job**, the name of the CI jobs as the identifier. In this figure, both flaky builds wasted CI time or exhausted CI resources for more than one hour. The bottom portion of this figure illustrates that the failing test (F) caused the failure of the flaky build, which we refer to as the **flaky test**. Although the flaky builds originated from two different reviews and projects, both pointed to the same flaky test. We define it as **cross-project flakiness**, which refers to *non-deterministic test failures that manifest across multiple projects in a software ecosystem*. It serves as a signal to the tests for their instability or non-deterministic behavior in this ecosystem. This particular flaky API test [52], asserts that the attached volume is still present and accessible in rescue mode, is the most frequently occurring in our data set, accounting for 201 instances. It has not only been flaky in the Cinder and Glance projects, but also in 14 other OpenStack projects, such as Nova, Devstack, and Tempest. According to the bug report,[4] Nova fails to detach the volume via Libvirt during test cleanup, despite retries. This flakiness is suspected to be attributed to changes introduced in Libvirt.

We investigate cross-project flakiness along flaky builds (coarse-grained level) and flaky tests (fine-grained level). In this example, the flaky test (F) can be consistently executed in the Neutron project (G). This test exhibits that *a deterministic status for other projects, while flakiness for one or more projects in a software ecosystem*, we define such inconsistent behavior as **inconsistent flakiness**. It serves as a signal to tests that the non-deterministic behavior may not be only inherited by the test logic (true behavioral inconsistency), but could also be triggered by configurations or environments in the project.

IV. Case Study Design

In this section, we describe the selection of our studied ecosystem and our approach to data preparation.

A. Studied Ecosystem

Our study aims to investigate the flakiness and its propagation across multiple interconnected projects at the ecosystem level. Thus, we need to collect testing logs from projects with frequently occurring builds on identical code (flakiness) in CI. Below, we illustrate our rationale for selecting OpenStack.

Popularity of the Studied Ecosystem. OpenStack is one of the largest open-source communities, with approximately 13 million lines of code contributed by around 12,000 developers. This ecosystem has been extensively studied within the code review context [34, 37, 73]. OpenStack comprises open-source projects supported by established organizations and companies (e.g., AMD, Microsoft, and Cisco), collaboratively developing a cloud computing platform.[5] Over the past decade, the OpenStack community has dedicated substantial effort to

[4]https://bugs.launchpad.net/nova/+bug/1960346
[5]https://www.openstack.org/community/supporting-organizations/

documenting and implementing code reviews, making it a valuable source of insights for similar communities.

Significance of the Studied Ecosystem. Red Hat [44] advocates for the "Upstream First" policy, while the OpenStack community [63] supports "Long-Term Support" and "Extended Maintenance" to maintain historically stable branches instead of terminating them, thereby providing a cost-effective and sustainable model for innovation in open-source platforms. These policies may significantly increase the complexity of maintaining the reliability and stability of interdependent upstream projects or branches within the OpenStack ecosystem. Such complexity highlights the importance of studying flakiness prorogation within OpenStack, as it can severely impact project stability and reliability.

Frequent Repeated Builds on Identical Code. Flakiness is characterized by non-deterministic testing outcomes. Unlike local testing, CI testing outcomes are transparent and observable by users. The OpenStack ecosystem employs the widely used Gerrit review tool for code reviews. Prior research [34] reported that developers in the OpenStack ecosystem unconstrainedly use the "recheck" command, a re-run technique to mitigate non-deterministic behavior. The recheck command is frequently invoked after the builds fail, with 55% of reviews being identified from 41,868 code review. However, less than half of the cases do not change the outcomes of failing builds.

B. Data Preparation

To prepare the OpenStack data for analysis, we first perform data collection, then build-related comments extraction, and finally, flaky build identification. We describe each step below.

Data Collection: We leverage the RESTful API[6] provided by the Gerrit tool to collect closed code reviews from OpenStack projects. Considering that logs from third-party testing in OpenStack are only retained temporarily—typically for one month [47], and potentially for shorter durations (with default retention set to 7 days [79])—we scrape the API on a daily basis to build this collection of one year. As shown in Table I, we capture 29,175 closed code reviews and 73,707 patch sets across 649 OpenStack projects from June 2023 to June 2024. Since we aim to investigate the interactions between developers and CI bots, we also collect review comments (which include the change log information) associated with each code review, yielding a total of 439,618 comments.

Build-Related Comments Extraction. We focus on Zuul CI, the primary pipeline-based project gating system in OpenStack, which facilitates the execution of tests and automated tasks triggered by code review events. We deem Zuul representative for our study since it is the most commonly employed CI system in code review. Indeed, 691 OpenStack projects [80] of 1,336 projects [46] (52%) use Zuul CI. We extract build-related comments by matching patterns (`[merge|build] [succeeded|failed]`) in the review comments posted by the Zuul bot. This approach enables us to identify 139,768 comments reporting build outcomes.

Flaky Build Identification. From the 139,768 collected Zuul comments, we select pairs of build invocations, which are repeated for the same patch set and have successful and failing outcomes. We classify the failing builds as flaky builds. The rightmost portion of Figure 1 labelled C illustrates an instance of detecting flaky builds in code reviews. Here, the build outcome for `287458[...]c6fde2` (the build invocation in the upper-right part of Figure 1) switched from failure to success after rechecking (the build invocation in the bottom-right part of Figure 1), thereby identifying it as a flaky build. The regular expression that we use to extract build information from comments can be found in the replication package. Applying that pattern-matching query to our review comment corpus, we detect 29,911 flaky builds across 1,651 unique Zuul jobs spanning 400 OpenStack projects.

TABLE I: Summary statistics of collected data sets.

# Studied Projects	649
Studied Period	June 2023 – June 2024
# Code reviews	29,175
# Review Comments (change log)	439,618
# Patch sets	73,707
Avg. - Med. - Max. # patch sets per CR	2.5 - 1 - 450
# Zuul comments	139,768
# Flaky jobs	1,651
# Flaky builds	29,911

V. Frequency of Cross-Project Flakiness (RQ1)

In this RQ, we investigate the extent to which cross-project flakiness occurs and the manner in which it is dispersed across the ecosystem.

A. Approach

To identify cross-project flakiness at a coarse-grained level (i.e., flaky builds), we examine whether a Zuul job exhibits non-deterministic behavior across multiple projects. For instance, the Zuul job "tempest-integrated-storage", shown in the upper-right of Figure 1, demonstrates flaky behavior in several projects, including `Cinder`, `Glance`, and `Zaqar`. Among the 29,911 flaky builds identified across 1,651 unique Zuul jobs, we categorize each job based on whether its flakiness recurs in the projects it was executed.

For comprehensive coverage in the OpenStack ecosystem, we aggregate builds from all 1,651 flaky Zuul jobs to analyze different levels of flakiness. Through the use of the Zuul API,[7] we collect additional build data, ensuring each flaky Zuul job included a sufficient volume of builds (specifically, at least 200) to accurately represent its occurrence and impact within the OpenStack code reviews. We subsequently identify whether a Zuul job has been executed in multiple projects.

To examine the frequency of cross-project flakiness, we monitor instances of flakiness by analyzing CI bot comments that indicate flaky jobs (e.g., C in Figure 1) within code reviews. This approach enables us to timestamp the occurrences of flakiness and trace their propagation over time. We identify whether flaky builds occurred across projects based on the data collected at that time for propagation analysis.

[6]https://review.opendev.org/changes/?q=status:closed

[7]https://zuul.opendev.org/api/tenant/openstack/builds

B. Results

In this paper, we present results with respect to observations, which are followed by the evidence that supports them.

Observation 1. **Cross-project flakiness affects half of OpenStack projects and is more frequent than non-cross-project flakiness.** Figure 2 shows that a total of 378 OpenStack projects, approximately 55% of all projects (691 projects that employ Zuul CI [80]), have been affected by cross-project flakiness. This high proportion of impacted projects suggests that flakiness is not merely an isolated issue, but instead permeates across the majority of OpenStack projects, affecting collaborative efforts and shared resources.

We find that cross-project flakiness not only affects many projects but also occurs more frequently than non-cross-project flakiness. In total, we identify 371 unique flaky Zuul jobs that had non-deterministic behavior across multiple OpenStack projects, compared to 1,280 flaky jobs limited to a single project. Although cross-project flaky jobs (18,353 flaky builds) represent only 29% of the non-cross-project flaky jobs (11,558 flaky builds), they contribute to nearly twice as many flaky builds. This indicates that cross-project builds are more prone to repeated failures, likely due to the shared dependencies and environmental conditions across projects. We observe that 44% of the cross-project flaky jobs had propagated to more than two projects, while the rest of the cross-project flaky jobs propagated to two projects.

Observation 2. **Cross-project flakiness has shown a consistent increase over time and grows faster than non-cross-project flakiness.** Over the observed time frame, the occurrence of cross-project flakiness has steadily increased, we find 52 projects where cross-project flakiness occurred in 2020; however, this number increased to 378 in 2024, a sevenfold increase after four years. This rising trend implies that as OpenStack projects increasingly rely on shared Zuul jobs, the probability of flakiness propagating across projects also rises, likely due to dependencies and shared testing infrastructure in OpenStack.

Figure 2 shows that cross-project flakiness demonstrates a faster growth rate over time compared to those not. This trend highlights the increasing complexity of managing CI workflows as projects evolve to use shared resources more extensively. The frequent occurrence of cross-project flakiness also emphasizes the need for robust cross-project coordination to address these issues effectively, as unmanaged flakiness in shared jobs can lead to considerable delays in code reviews and inefficiencies across dependent projects.

> Cross-project flakiness affects 55% of OpenStack projects and is implicated in twice as many flaky builds as those not. Cross-project flakiness has grown more prevalent over time, likely due to shared dependencies and environmental inconsistencies across projects.

VI. Relationship Between Test Scopes and Flakiness Levels (RQ2)

In this RQ, we investigate the influence of test scopes (e.g., unit and full-stack tests) on the dispersion of flakiness.

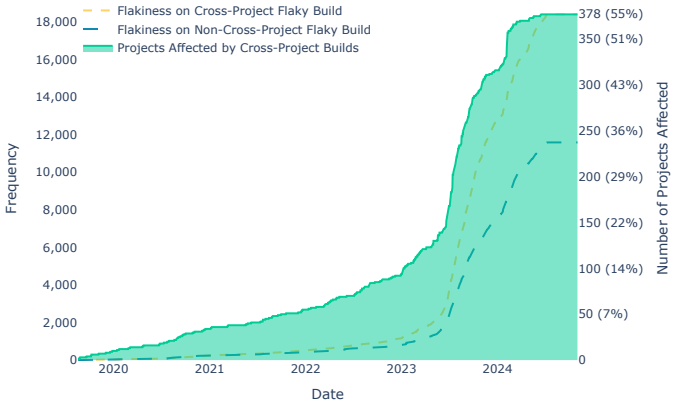


Fig. 2: Propagation of flakiness over time.

A. Approach

Test Result Collection. For each flaky build identified in Section V, we retrieve the associated build log URL, containing a unique identifier (a.k.a., UUID) to access specific job outputs (e.g., `job-output.txt`) and test results (e.g., `testr_results.html`).

Our access to build logs is constrained by the log retention policy, which varies among the studied projects. Out of a total of 29,911 flaky builds, 13,140 were invoked during our retention period of one year. Despite these constraints, we successfully retrieve `testr_results.html` files from 3,369 flaky builds, spanning 370 unique jobs. We filter out data from projects that no longer exist at the time of data collection in two cases: (i) unavailable job logs, and (ii) job records no longer existing (e.g., `heat-functional-yoga`).

Flaky Test Identification. From successfully collected test results, we identify specific test cases that had failed as flaky tests. Each flaky test is identified by the combination of its fully-qualified class and test method names, e.g., `tempest.api[...]RescueTest` and `test_stable[...]volume_attached[...]` in Figure 1, respectively. The first segment (e.g., `tempest`) in the class name indicates the root package name. Together with the test method name, it ensures tests with this identifier correspond to semantically equivalent code and prevent naming collisions across OpenStack projects. This identifier is also used by the test runner (`stestr`) [67, 68] for reporting results and OpenSearch [48], adopted by the studied community for searching OpenStack testing and building logs.

In total, this process yields 11,506 flaky tests, which span 57,124 flaky instances (executions).

Test Scope Classification. To understand the nature of these flaky tests, we systematically identify different scopes of 11,506 flaky tests. Our approach extracts all segments except the first one (e.g., `tempest`) from the dot-delimited fully-qualified class name for keyword-matching. The test scopes and the corresponding keywords for each test scope are built based on the OpenStack general [56] and project-specific [57, 58] documentation to reflect the testing design in the OpenStack ecosystem and avoid matching ambiguous identifiers. For example, the Neutron documentation [57]

TABLE II: Distribution of flaky tests and instances by scope and flakiness level in OpenStack projects.

	Category	Definition	# Tests	# Instances	Avg. Executions
Test Scope	Unit	Tests individual units of code in isolation, ensuring correctness during the development phase, involving the term "*unit*" in its identifier.	7,202	30,550	2,845
	API	Evaluates the functionality, performance, and security of APIs to ensure compliance with specifications, containing "*api*" in its identifier.	1,285	10,599	3,617
	Resource Management	Manages resources and configurations required before all tests or releases resources after all tests in a test class. These tests typically start with "*setUpClass*" or "*tearDownClass*".	636	8,414	37
	Scenario	Simulates end-to-end functionality to validate integration points between OpenStack services, containing "*scenario*" in its identifier.	383	3,065	1,328
	Functional	Assesses broader functionalities of an OpenStack project, verifying correct outputs for various inputs, including "*functional*" in its identifier.	1,230	1,963	488
	Full Stack	Verifies interactions between OpenStack components to ensure they operate correctly in combination, including "*fullstack*" in its identifier.	85	293	885
	Undefined	Any test that does not fit the above categories.	685	2,240	639
Flakiness Level	Shared-Partial Trigger	Flakiness appears across multiple projects, with flaky tests triggered only by some of the affected projects.	703	16,976	4,453
	Uniform Multi-Project	Flakiness spans multiple projects, with the same set of projects consistently triggering these flaky tests.	832	3,949	333
	Cross-Trigger	Flakiness is present in a single project, yet its test is also triggered by other projects.	402	1,926	1,955
	Project-Isolated	Flakiness is confined to a single project, with the associated test triggered only within that project.	371	1,714	162
	Failed to obtain	The build logs are not available.	95	490	-

explains that "Unit test modules should have the same path under neutron/tests/unit/ as the module they target has under neutron/." These test scopes and keywords (available in the top portion of Table II) are iteratively and manually refined by the authors to minimize undefined cases (e.g., "setUpClass"). For multiple segments matching, we prioritize classification from longer, less common test scopes (Full Stack) over shorter, more common scopes (API and Unit).

Flakiness Level Classification. To classify flakiness levels, we identify how flakiness behaved across different OpenStack projects. We categorized each flaky test into one of four levels based on the definitions in the bottom portion of Table II.

To automate flakiness categorization, we collect test results from builds obtained in RQ1. For example, we retrieve the test results of the `tempest-integrated-storage` job, shown in the upper-right of Figure 1. In this example, three OpenStack projects (`Cinder`, `Glance`, and `Zaqar`) executed the flaky test identified in this job. If this flaky test only appears flaky in `Glance`, we categorize it as *Cross-Trigger* flakiness since the flakiness is limited to one project. When the flakiness occurred in both `Cinder` and `Glance`, we classify these as *Shared-Partial Trigger* flakiness since the flakiness propagated more than one project, but not all projects run it. If the flakiness was observed in all three projects, we classify this as *Uniform Multi-Project* flakiness since the flakiness propagated across all projects. Conversely, if flakiness was only observed in `Glance` and the test was exclusively run by `Glance`, this is classified as *Project-Isolated* flakiness.

For the two phenomena we discovered, cross-project flakiness is a combination of *Shared-Partial Trigger* and *Uniform Multi-Project* flakiness, while inconsistent flakiness results from a combination of *Cross-Trigger* and *Shared-Partial Trigger* flakiness.

B. Results

We plot the relationship between test scopes and levels of flakiness by using the parallel sets [27]. Parallel sets are variants of parallel coordinates in which the width of lines that connect sets corresponds to the frequency of their co-occurrence. To plot more accurate results in identifying high-impact flakiness within OpenStack, we **exclude flaky tests that appeared in only a single patch set**. This filtering yields 2,403 flaky tests from 11,506 flaky instances, each occurring in multiple patch sets.

Observation 3. **Unit tests are the most likely to exhibit flakiness.** The top portion of Table II illustrates the distribution of flaky tests and their instances across various test scopes in OpenStack projects. Unit tests account for the largest amount of flaky tests (7,202) and instances (30,550). According to the OpenStack documentation [56]: "Most projects have a large number of unit tests and expect to have near-complete code coverage with those tests." Thus, it is not unexpected that the large quantity of unit tests overall accounts for a large number of flaky tests. In our data set, we find the most commonly flaky unit test [53], exhibiting flakiness 72 times in the Nova project. The test verifies that the database schema after migrations is in sync with the expected models in the code.

API tests are the second most frequently implicated, with 1,285 flaky tests and 10,599 instances. Flakiness in API tests typically stems from their reliance on external components and third-party services, which may not always be stable or available. Furthermore, the intrinsic complexity of cloud environments, characterized by interacting components operating concurrently and often asynchronously, exacerbates this issue. Such environments can induce non-deterministic behavior, particularly when interactions are not managed or simulated within test scenarios [43]. We find that the API test [52] that is most frequently flaky impacts 14 OpenStack projects. The test evaluates the functionality of uploading a volume to create

an image using the API.

A substantial proportion of flaky instances are associated with resource management tests. These tests orchestrate resources under conditions shared across multiple tests within the same test class, ensuring a consistent starting environment and proper cleanup after tests to prevent side effects on subsequent tests. Flakiness in these tests is likely due to concurrency issues or dependencies arising from the shared state or resources.

To reduce the influence of flaky test execution frequencies on our observations, we estimate these frequencies across OpenStack by computing the average number of executions for each test scope in our collected test results, as shown in Table II. While we observe more flaky unit tests in our data set, their average execution frequency (2,845) is lower than that of API tests (3,617), which further supports that Observation 3 is not caused by frequent executions of unit tests in CI.

Observation 4. **Flakiness in cross-project tests is more prevalent than in single-project tests.** The bottom portion of Table II presents the distribution of flakiness levels, showing that *Shared-Partial Trigger* and *Uniform Multi-Project* flakiness are the most prevalent, accounting for 16,976 and 3,949 instances, respectively. This trend indicates the spread of flakiness affecting tests executed across multiple projects, accounting for 1,535 $(703 + 832)$ tests under cross-project flakiness phenomenon. The *Cross-Trigger* flakiness is detected in 402 tests. Similar to *Shared-Partial Trigger* flakiness, they belong to the inconsistent flakiness phenomenon, which affects a total of 1,105 $(703 + 402)$ tests. In Section VII, we further examine why specific flakiness types predominantly impact one or a few projects. Conversely, *Project-Isolated* and *Uniform Multi-Project* flakiness occur under ideal circumstances, either affecting only one project or all projects uniformly.

Observation 5. **Beyond functional and full-stack tests, test scopes demonstrate correlations with cross-project flakiness, as unit tests, intended to validate individual components, exhibit flakiness across multiple projects.** In Figure 3, functional and full-stack tests are strongly associated with *Project-Isolated* and *Cross-Trigger* flakiness, accounting for 46% and 86% of cases, respectively. Flakiness in functional tests frequently remains confined to a single project. As stated in OpenStack's documentation [56], "Functional tests validate a project in operation (e.g., if the project is an API server, the server will actually be running), though interactions with **external components are kept to a minimum**."

A substantial percentage (70%) of unit tests exhibits *Uniform Multi-Project* flakiness, indicating that for all OpenStack projects where these unit tests were executed, the flakiness occurred consistently. For example, `test_servers` [50] test, used in projects such as `Glance` and `Cinder`, has spread flakiness due to its reliance on `OpenStackSDK` to interface with the Nova API. By contrast, API tests (64%) and scenario tests (41%) show *Shared-Partial Trigger* flakiness, where flaky behavior appears in multiple projects but only within a subset of them. This is expected, as they gate commit integrations into OpenStack, which may behave differently based on environmental contexts. Similarly, 61% of resource management tests follow this pattern, suggesting possible misconfigurations or poor resource management.

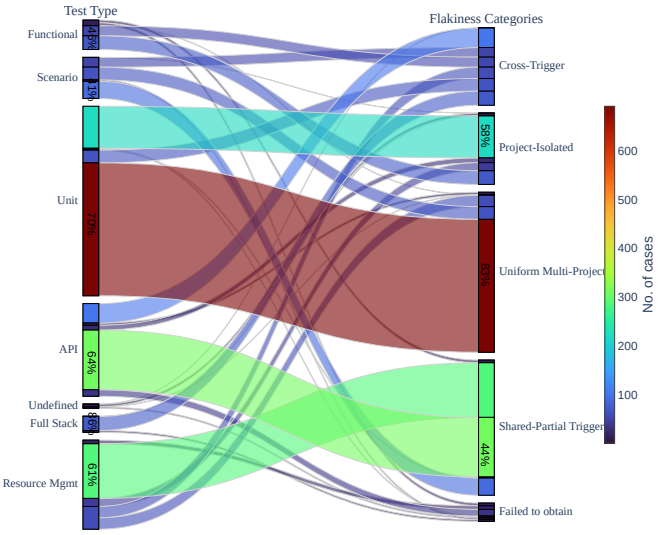


Fig. 3: Parallel sets between test scopes and flakiness levels in OpenStack projects.

> About half of API tests and scenario tests are flaky across projects, likely due to similarity to integration-level testing. Substantial unit tests are evaluated with flakiness in multiple projects, indicating potential misalignments between the test design and its intended scope. More than half of resource management tests show inconsistent flakiness, suggesting misconfigurations or issues with resource management within the test design.

VII. Causes of Inconsistent Flakiness (RQ3)

In this RQ, we qualitatively investigate why flakiness occurs in specific OpenStack projects rather than affecting all projects that execute the same flaky test. We analyze the root causes of inconsistent flakiness instead of cross-project flakiness to provide a control condition, since the test logic of these inconsistently flaky tests is observed to be stable in the "control" project, the non-deterministic behavior could be attributed to possible project-specific configurations rather than intrinsic non-deterministic behavior, which has been widely analyzed in the single-project context [15].

A. Approach

We employ an open coding approach [10] to classify inconsistently flaky tests. Open coding, a qualitative analysis method, involves classifying the artifacts under investigation (flaky tests in this case) according to emergent concepts. Following this initial coding, we apply open card sorting [42] to organize low-level codes into higher-level categories. Below, we detail our coding saturation, coding strategy, and card-sorting procedures.

Coding Saturation. This RQ focuses on 1,105 inconsistently flaky tests spanning 18,902 $(16,976 + 1,926)$ instances

or test executions (a flaky test can manifest its flakiness behavior multiple times); specifically, *Cross-Trigger* and *Shared-Partial Trigger* flakiness manifests in a subset of projects but not all of them. Given the impracticality of coding all 1,105 tests for over ten thousand instances, we code a subset of these flaky tests to enable analytic generalization and reach theoretical saturation [16] for exploring the underlying cause of this novel phenomenon in the OpenStack ecosystem.

To discover as complete of a list of reasons for inconsistent flakiness as possible, we strive for theoretical saturation [16] to achieve analytic generalization. We set our saturation criterion to 10: the first two authors continue to code randomly selected flaky tests until no new codes have been discovered for 10 consecutive flaky tests. Since a flaky test could manifest many flaky instances, we adopt a relatively small saturation criterion. Furthermore, a single flaky test may have multiple labels.

We produce an initial set of codes after the first 30 flaky tests. Based on this set of codes, the first two authors independently code another 30 flaky tests and then calculate Fleiss's kappa agreement [18] of this iteration among all raters. The kappa agreement of this iteration is 0.726 or "Substantial" agreement [71]. Based on this encouraging result, the remaining data is then coded by the first author. Finally, we reach saturation after coding in total of 110 inconsistently flaky tests, spanning 671 test executions. Since codes that emerge late in the process may apply to earlier reviews, we perform another pass over all of the flaky tests to correct miscoded entries and test the level of agreement of our constructed codes. These 110 coded flaky tests span all test scopes defined in RQ2 and account for 62% of projects exhibiting inconsistent flakiness.

Coding Strategy. Our coding strategy aligns with OpenStack's official documentation [51] on handling test failures. First, we review available CI logs and error messages to identify symptoms of flaky tests. Given that flaky test causes can be non-deterministic, we also examine relevant inline comments in the code reviews. Finally, we search related bug reports based on the test symptoms to identify possible causes. This process is extensively conducted on triangulating CI logs, error messages, and bug reports for 671 test executions.

Card Sorting. We apply open card sorting to construct a taxonomy of factors that contribute to the occurrence of flakiness in one software project but not in others. Open card sorting produces generate general categories from our low-level codes. The open card sorting includes two steps. First, the coded cases are merged into cohesive groups that can be represented by a similar subcategory. Second, the related subcategories are merged to form categories that can be summarized by a short title.

B. Results

Observation 6. **Inconsistent flakiness across projects is primarily caused by event-related issues, such as server downtime or temporarily unavailable resources, but we also observe dependency issues of the patch, and inconsistent build configurations.** In Table III, *Event-related flakiness* is the most frequently occurring reason for inconsistent flakiness. These inconsistencies can stem from race conditions in CI or dependency issues from the patch. Interestingly, a small number of inconsistencies were linked to build configurations. Only one specific flaky test occurred due to an unordered collection comparison in the `assertEqual` function.

TABLE III: Definition and frequency of reasons for inconsistent flakiness.

Reason	#	Definition
Event-related flakiness	**98**	Flakiness in this category is caused by external or transient events impacting the stability of tests across different projects. These events (race conditions in CI) may be unpredictable and are often resolved in the short term.
Backend server down	25	
Temporarily unavailable resource	21	
Short-term job fixes	17	
Networking issues	14	
Unstable job within period	13	
Simultaneous reading conflicts	8	
Dependency-related flakiness	**23**	Flakiness caused by dependencies on external components or project transitions, affecting tests differently across projects due to versioning, updates, or reliance on specific branches.
Bump issues	17	
Dependent issues	2	
Supporting branch transition	2	
Dependent with test change	2	
Configuration-related flakiness	**23**	This type of flakiness is due to configuration issues within the testing environment or infrastructure, leading to inconsistencies in test outcomes across projects.
Slow deprovisioning of virtual machines	19	
Temporary server disabling	4	
Other	**1**	An outlier category for flakiness caused by a miscellaneous or unordered collection issue not fitting the main categories above.

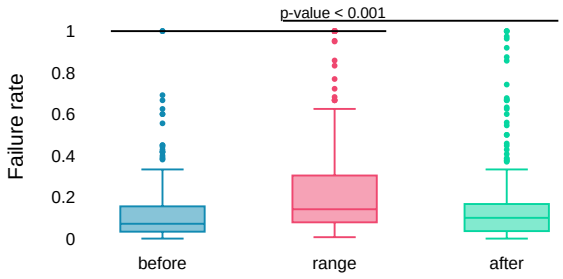


Fig. 4: Failure rates of each Zuul job before, within, and after flaky range for inconsistent flakiness.

In total, 98 cases exhibit inconsistent flakiness primarily due to event-related causes. For instance, flakiness was observed when the backend server was down at a particular time in an API test. A related bug report [62] stated:

> I looked at the logs from link in comment #1 for the request to create volume [...]. The request was serviced by the cinder-scheduler running on controller-2, and the logs show the RBD backend was down at that time.

Another example of event-related flakiness occurred in a scenario test due to temporarily unavailable resources. The error log indicated:

> Traceback (most recent call last): File "[...]", line 521, in [...] BlockingIO-Error: [Errno 11] Resource temporarily unavailable

This type of flakiness tends to occur repeatedly within a short timeframe. We then investigate whether the failure rates of the flaky jobs for these flaky tests differed significantly from the flakiness possibly caused by race conditions in CI. We define the flaky range as the duration from the first identified instance of flakiness to the latest recorded instance in our data set for each flaky job. This range provides perspective on the persistence of flaky behavior over time and across projects. We then examine differences in failure rates within, before, and after the flaky range of each job by performing a Mann-Whitney test [36] (two-tailed, unpaired, $\alpha = 0.05$). To measure the effect size, we also apply Cliff's δ [11], a non-parametric effect size measure. Effect size is characterized as *negligible* if $\delta < 0.147$, *small* if $0.147 \leq |\delta| < 0.33$, *medium* if $0.33 \leq |\delta| < 0.474$, *large* if $0.474 \leq |\delta|$ [66].

Observation 7. **Inconsistent flakiness caused by race conditions in CI has significantly increased the failure rates.** Figure 4 indicates that failure rates of flaky jobs within the flaky range are significantly higher than before and after the flaky range. The Cliff's δ of the before and within the flaky range is *medium*. We find a *small* effect between within and after the flaky range. This confirms our qualitative exploration, where the principal cause happened in a short timeframe.

Beyond event-related issues, 23 cases exhibit inconsistencies related to dependency issues. A notable example involves a version bump issue, where developers rechecked the patch (triggering the failed job to execute again on the Gerrit platform) with an inline comment for a flaky full-stack test:

> recheck requirements bump merged

Inconsistently flaky tests may also be caused by the temporary disabling of test cases when a dependent project transitions to an unmaintained branch [49]. For instance, a developer rechecked the patch set with the following comment to another review:

> recheck - [...] 906582 has merged
>
> $\Downarrow$ *Review* #906582*'s Description*
>
> [requirements] Transition Yoga to Unmaintained. With the new TC resolution the community replaces Extended Maintenance with Unmaintained status. So this patch transitions stable/yoga branch to Unmaintained [...]

We also find flakiness can result from differing CI configurations (23). For example, one developer rechecked the patch:

> recheck Fixed IP address [...] is already in use on instance likely caused by slow vm

In such cases, the resource is likely occupied by other processes, as suggested by the error message:

> Target share type is still in use.

Solutions for these issues include limiting thread concurrency in CI environments [54] or increasing the swap memory size [55]. Another potential solution is silencing services in the CI configuration:

> recheck [...] 922902 was merged
>
> $\Downarrow$ *Review* #922902 *Description*
>
> Disable openstack-cli-server. The server does not support switching credentials well and causes problems with resources created for [...] tests.

> Results in RQ3 suggest a need to standardize CI configuration as well as dependency management across the ecosystem. The primary cause of inconsistent flakiness is unanticipated event-related issues like server downtime. Inconsistent flakiness can also arise from dependency-related issues and configuration issues in CI setups, indicating that CI setting variations can also lead to flakiness.

VIII. Time Wasted Due to Cross-Project Flakiness

In Section V, we find that cross-project flakiness is a prevalent phenomenon at a coarse-grained level. To further measure the subsequent impact of cross-project flakiness, in this section, we study the time it wastes during code review. Specifically, our goal is to compare the waste caused by cross-project flakiness to that of flakiness occurring within a single project. In part C of Figure 1, flakiness is also associated with slower code review decisions and the consumption of additional build resources. In total, these add a delay of 2h 07m 05s. To more broadly quantify the impact of cross-project flakiness, we measure this additional build execution time across the 29,911 identified flaky builds. We perform Mann-Whitney tests to assess the significance of the difference between cross-project and non-cross-project flaky jobs.

Observation 8. **Cross-project flakiness generates more time waste than flakiness limited to a single project.** We find that flakiness has cumulatively wasted 1,156 days on code reviews, averaging 0.93 hours of additional time per flaky build. Figure 5 shows that cross-project flaky builds have a higher median of time waste than non-cross-project flaky builds. The Mann-Whitney test confirms this with a significant difference ($p < 0.001$). The effect size for this significance of the difference is *small*, i.e., $0.147 \leq |\delta| < 0.33$. It indicates that cross-project flakiness is associated with slower completion of code reviews (due to the re-execution of unreliable CI jobs) than flaky tests that occur within a single project. This also suggests that general flakiness prolongs code review cycles in OpenStack. If flakiness propagates across multiple projects, it produces significantly more time and resource waste.

IX. Developer Feedback

To gain deeper insights into cross-project and inconsistent flakiness, we conduct a developer questionnaire[8] to understand how developers perceive these issues. Our goals are to: (i) gain perspective on the practitioner experience with these challenges and how these problems are typically addressed, (ii) gather feedback on the causes of inconsistent flakiness, and (iii) explore existing tool support and opportunities for

[8]The questionnaire was approved by the Kyushu University, Japan ethics board (2024-26) and is available at https://tinyurl.com/croflakiness.

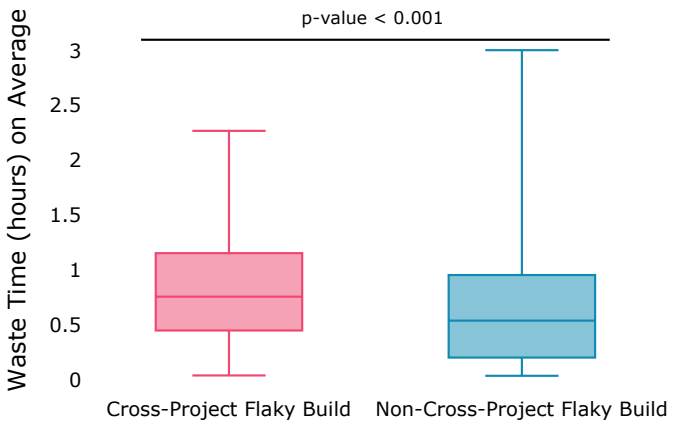


Fig. 5: Time wasted (hours) of each Zuul job.

TABLE IV: Respondents' role and their experiences in OpenStack as well as with cross-project and inconsistent flakiness.

ID	Role	Experience		
		OpenStack	Cross-project	Inconsistent
R_1	Contributor	3-5 years	No	Yes
R_2	Core developer	More than 5 years	Yes	Yes
R_3	Core developer	More than 5 years	Yes	Yes
R_4	Contributor	More than 5 years	No	Yes
R_5	Contributor	More than 5 years	Yes	No
R_6	Core developer	3-5 years	Yes	Yes
R_7	Contributor	More than 5 years	No	Yes
R_8	Project maintainer	More than 5 years	Yes	Yes
R_9	Contributor	More than 5 years	Yes	No
R_{10}	Contributor	More than 5 years	Yes	Yes
R_{11}	Core developer	More than 5 years	Yes	Yes
R_{12}	Consultant for contributors	1-2 years	No	Yes
R_{13}	Contributor	1-2 years	No	Yes
R_{14}	Contributor	More than 5 years	Yes	Yes
R_{15}	Contributor	1-2 years	Yes	Yes

developing new tool support. We distributed an open invitation to participate in our questionnaire via the OpenStack discussion mailing list and reached out to 833 developers who had participated in the studied code reviews. We accepted questionnaire responses for one month and received valid responses from 15 developers.

Table IV provides an overview of the demographics of our respondents. Among the 15 respondents (R_1–$_{15}$), ten reported having over five years of experience contributing to OpenStack projects. Ten participants are contributors or consultants for contributors, while five are core developers or project maintainers. We analyze open-ended questionnaire responses using a card-sorting method. Below, we summarize the key findings, with the full questionnaire design available in our online appendix.

Cross-Project Flakiness. Ten respondents indicated that they had encountered cross-project flakiness in OpenStack. In terms of addressing cross-project flaky tests differently from those in a single project, two respondents noted an increase in complexity due to the need for substantial developer collaboration across multiple teams. For example, R_5 explained that "Less likely to engage due to higher burden (having to engage multiple teams)," whereas R_{10} emphasized the challenges of collaboration and obtaining feedback on such flakiness, particularly from the perspective of a newcomer:

TABLE V: Causes of inconsistent flakiness and their debugging difficulty from respondents.

Causes	Distribution	Debugging difficulty
Event-related	42.9%	
Dependency-related	28.6%	
Configuration-related	14.3%	
Others	14.3%	

The darker bars suggest a greater difficulty level in the debugging process.

> *"I personally do a recheck, I am **new to contributing** and I find it **difficult to get help from the community** so I often just **ignore it or try to keep away from contributing**. I have reviews up which I **never get feedback on**, also **asking via IRC does not help** so I **do not really want to engage in fixing tests or systems**."*

Three respondents mentioned common causes, such as race conditions, concurrency, and infrastructure-based challenges on the testing platform (R_7, R_9, and R_{14}). Two respondents explained that they often do not take further actions beyond rechecking and waiting for a subsequent build. Two respondents suggested verifying shared dependencies, environments, and test fixtures in different projects (R_{12} and R_{13}).

R_8 remarked, "I prefer to view OpenStack as a **collective effort**, so failures in any one component are a failure in OpenStack as a whole." Indeed, OpenStack provides OpenSearch, a community-maintained cluster that monitors failing tests and jobs. R_{12} similarly highlighted the importance of reviewing environments and test fixtures to understand why a test is flaky. R_1 added, "Cross-project is a sign of **incorrect structure**."

Inconsistent Flakiness. The 13 respondents reported encountering inconsistent flakiness. In terms of causes beyond those already identified in Section VII, R_{12} described "test fixtures from different vendors running different firmware versions." Table V shows that other responses provide support for issues identified in Section VII, such as networking issues.

Developers expressed diverse opinions about the difficulty of debugging each cause. Event-related causes are generally perceived to be the most challenging, potentially due to that external factors can be unpredictable. R_{12} also cited test fixtures from different vendors as especially difficult to debug.

Possible Tool Support. Ten respondents believed that tooling could help mitigate inconsistent flakiness. For example, R_3 pointed out that tools can overcome this organizational or infrastructure barrier. Four other respondents have no opinion on it, while R_{10} argued the opposite position, stating that "I honestly do not [agree] here, I think more tests should be [set up and] verified, but I also believe that it must not be a burden to enter the contributor community, which I currently really [feel] like [it is]." Similarly, R_8 described why they felt that prior tool-based attempts to mitigate inconsistent flakiness did not succeed:

> *"We've had useful tools for this in the past: scripts which bisect dependency version changes, Bayesian analysis of test*

> *log messages to correlate with job success/failure, curated repositories of error conditions mined from software logs mapping to specific unresolved bug reports, databases of granular integration test success/failure within and across projects... the common thread is that tools* ***require maintenance and attention to their outputs****, i.e. dedicated time prioritized by contributors, a commodity in short supply for most open source projects (including OpenStack). Ultimately these tools, while fun to create, have fallen into disrepair and been abandoned due to* ***lack of enough people caring for them over the long term****."*

R_6 noted that the absence of full-time paid developers focused on build processes may impede sustained tooling.

> *There's full time [paid] people needed to consistently work on, fix and improve the [OpenStack] build process. I'm currently not aware of anyone doing this full time, [e.g.,]* ***most people spend a few hours per week on the CI, but there is no dedicated person working on it full time****, as far as I know.*

Five respondents also offered specific suggestions for tool support. R_5 proposed an error-tracking and monitoring tool such as Sentry,[9] which is similar to the tools described by R_8. R_{12} highlighted the value of enforcing consistent test environments across projects to deal with flakiness in general. R_7 recommended specialized tooling for event-related flakiness, while R_6 suggested verifying or invalidating build configurations across projects.

While participants such as R_{15} agreed that rechecking practices could be improved with automation, R_8 raised concerns about blind rechecking, emphasizing that shared ownership benefits OpenStack overall:

> *"The CI/CD system OpenStack relies on (Zuul), is constantly evolving and improving, and is the most effective place we've found to collaborate on implementing or integrating what solutions and mitigations we do come up with. That's why it came into existence in the first place. I would encourage contributors to* ***pay attention to job results and learn to investigate failures they don't understand****. We've had far too many cases of* ***nondeterministic bugs introduced into projects because people just kept blindly rechecking*** *until they got lucky with successful results enough times in a row to get a [chance] to merge, at which point it becomes everyone's problem (including users when that nondeterministic behavior eventually ends up in their production deployments). Even if the failure seems to be unrelated to the project your change is touching, fixing bugs in OpenStack benefits OpenStack as a whole, which includes that sub-component project."*

X. Threats to Validity

Below, we describe the threats to the validity of our study.

Construct Validity. To determine if flaky jobs occur across multiple projects (RQ1) and, at a more granular level, whether a flaky test exhibits different levels of flakiness (RQ2), we analyze the logs of 1,651 unique Zuul jobs. As such, it is possible that jobs and tests may be misclassified due to insufficient collection, e.g., executions of a flaky test are not captured in specific projects. To combat this threat, we collect at least 200 runs for each flaky job, obtaining 87,711 valid test result files, which reduces the likelihood of such types of misclassification. In RQ2, our test scope classification is based on community-defined labels embedded within the dot-delimited fully-qualified class name of tests [56–58], rather than strict definitions of test scopes. The choice is determined to reflect the practical observation in the OpenStack ecosystem.

Content Validity. This study does not manually classify the entire data set or a fixed-size random sample of flakiness inconsistencies, posing a risk of undiscovered cause categories. Instead, we strive for theoretical saturation [16] to achieve analytic generalization, a broadly adopted approach in software engineering research [22, 65, 74]. To combat this threat, we conduct eight iterations and achieve saturation after coding 110 inconsistently flaky tests and reviewing 671 flaky instances to triangulate CI logs, error messages, and bug reports.

Internal Validity. In RQ2, we investigate the relationship between test scopes and flakiness levels. The observed distributions may be attributed to the disproportionately high test or execution frequency of specific test scopes (e.g., unit tests) as well as disparities in CI job configurations across the OpenStack ecosystem. Given the difficulty of tracking the total volume of test executions across the entire OpenStack ecosystem, we report the average number of executions for each test scope. These values are derived from occurrences in our data set and may not accurately represent executions across the OpenStack ecosystem. Furthermore, we also mitigate this potential bias by reporting relative numbers rather than absolute numbers for Observation 5.

Since the causes of inconsistent flakiness rely on manually coded data in RQ3, potential miscoding may arise due to the subjective nature of schema interpretation and experience in OpenStack, as well as in searching for test symptoms on their bug report system. To mitigate this, we conduct a developer questionnaire and our responses of the respondents support these causes (Table V). In addition to those we identified in RQ3, only one other potential cause was recognized: text fixtures originating from diverse vendors. While we validate these causes in the developer questionnaire, we did not cross-validate them across the broader OpenStack ecosystem nor perform a granular, case-by-case validation with OpenStack developers. Consequently, there is a risk that specific instances may have been misattributed to incorrect bug reports. Moreover, without performing cross-validation, we can not distinguish which causes were the true behavioral inconsistency that may be crucial for repairing inconsistent flakiness. Future work could partner with OpenStack maintainers to identify and validate true behavioral inconsistency of inconsistent flakiness.

External Validity. Our study focuses exclusively on the OpenStack ecosystem, which may limit the applicability of our results to other ecosystems. We opt to emphasize internal validity by analyzing 94% of OpenStack projects that employed Zuul CI, providing substantial coverage within this domain.

To mitigate it, we examine the analytic generalizability [76]

[9]https://sentry.io/

of the two identified phenomena (cross-project flakiness and inconsistent flakiness) by extending our analysis to another ecosystem, Zuul, a widely adopted CI/CD system. Although Zuul was originally developed for OpenStack projects, it has since become independent of OpenStack, with open-source and proprietary users outside of the OpenStack ecosystem.[10] Thus, Zuul forms its own, albeit smaller, ecosystem, which serves as a valuable case to validate the phenomena we identified in this paper. Moreover, Zuul CI is also integrated into the code review process of the Zuul ecosystem, allowing us to reuse the same approach outlined in Sections IV–VI. In total, we collect 757 closed Zuul reviews across six projects from January 2025 to November 2025. For cross-project flakiness, we identify one job (`zuul-quick-start`) out of 53 Zuul jobs exhibiting non-deterministic behavior across multiple projects. For inconsistent flakiness, we found three flaky tests out of 51. Moreover, we discover that 46 of them are unit tests, which is consistent with Observation 3. These results collectively confirm the applicability of our results in Zuul. In future work, we plan to extend the investigation to other larger ecosystems to further enhance generalizability.

XI. Related Work

In this section, we position our work with respect to the literature on flaky test identification, Continuous Integration (CI) in modern code review, and flaky tests in CI.

Flaky Test Identification. Re-running tests is often employed to pinpoint tests that non-deterministically pass and fail on the same code. Tools such as DeFlaker [4] leverage differential code coverage to detect failing tests that did not execute newly changed code (likely flaky failures). Similarly, iDFlakies [30] reruns tests in different orders to discover both order-dependent and non-order-dependent flakiness. These automated rerun detectors uncover flaky tests that may not be caught in a single attempt.

Due to the time-consuming nature of reruns, researchers explored static analysis and machine learning for predicting flaky tests. The FlakeFlagger tool [3] uses test smells and metrics to predict flakiness without test execution, attaining an F1 score of 86%. Flakify [17] uses a pre-trained model and source code features to achieve impressive cross-validation performance, being especially adept at identifying flaky tests in unseen projects. While these models are promising, all flaky tests cannot be identified using code structure alone.

Research on Pull Requests (PRs) [29], commits [33], and bug fixes [69] has been conducted to explore flaky tests in practice. For example, Lam et al. [29] examined PRs that fixed flaky tests in six Microsoft projects, observing recurrent issues like asynchronous waits or race conditions, and incomplete fixes. This indicates that identifying flakiness is challenging; developers may believe that a test is stable after changes, but the test remains flaky. It warns that relying solely on developers' reports of flaky test fixes can be misleading. Overall, identifying flaky tests often requires combining multiple signals (rerun results, static code patterns, historical failure data) to differentiate genuine failures from false alarms.

[10]https://zuul-ci.org/users.html

CI in Modern Code Review. Rahman and Roy [60] analyzed the impact of automated builds on code reviews, observing that passing builds had a large influence on code review participation. To understand how developers use the outcome of CI builds during code review, Zampetti et al. [77] empirically investigated the interplay between PR discussion and the use of CI. Their study indicated that successful CI builds positively affect code review results compared to failed ones, while questionnaire findings highlighted that CI adds complexity to code review configuration and maintenance. Bernardo et al. [5] found that projects deliver merged PRs more quickly after adopting CI, and increasing the number of PRs submitted for projects slows their delivery time. Other researchers also found that: (i) adopting CI could reduce commenting noise as a silent helper [9]; (ii) difficulty in resolving CI failures is one of the reasons contributors abandon their PRs [25]; (iii) longer CI build time can prolong the review duration, but passed CI could be reviewed in a shorter time.

Flaky Test in CI. In CI environments, flaky tests pose challenges, and past studies have explored CI data to detect such tests. One method observes restarted builds; if developers rerun a failing build without modifying the code and it passes, a flaky CI step is to blame. An empirical study [14] of Travis CI found developers manually restarted 1.72% of all builds, totaling over 56,000 restarts, with nearly half succeeding upon retries. Maipradit et al. [34] shifted focus to OpenStack, where the "recheck" command allows reruns of CI jobs in code review, showing that 55% of OpenStack reviews involved repeated builds, with 42% altering test results, exemplifying flakiness. These rechecks consumed the equivalent of 187.4 CPU years and delayed code review decisions by a total of 16.8 years, excluding justified repeats. Similar issues exist in proprietary CI, such as at Google [41], where 1.5% of tests are flaky and 16% show flakiness. At Google, 84% of post-submit test failures (pass to fail), were due to flakiness instead of code changes, meaning five out of six such failures were false alarms. This wastes failure diagnosis effort, desensitizing developers who may overlook genuine issues. Olewicki et al. [45] reported that 31% of the commits had at least one manually rerun build job, and 15% of the build jobs were rerun at least once at one of the world's leading AAA game producers. Leinen et al. [32] discovered that at least 2.5% of the productive developer time is spent on flaky tests in industrial CI; however cost for rerunning tests is negligible to the developers' efforts. Studies in open source and industry CI settings have demonstrated that flaky tests incur high costs through wasted resources, delayed actions, and time spent diagnosing unreliable test signals.

Unlike prior work, our study focuses on flakiness that manifests code review. We complement the prior work that has focused on open-source CI services (e.g., Travis CI [14]) and proprietary CI systems (e.g., Google's internal pipeline [41]) by studying an infrastructure-based open-source ecosystem (OpenStack). We also adopt a more fine-grained perspective. Prior work on restarted builds in Travis CI examined how flaky builds prolong code review time, whereas we quantify the additional build time incurred by these flaky builds. Similarly, studies of repeated builds in OpenStack [34] have often been

coarse-grained, treating each rerun as a single unit, In contrast, our analysis is fine-grained, drilling down to individual failing tests within repeated failing builds. Finally, unlike earlier research that primarily investigated root causes of flaky tests, we broaden the scope by examining cross-project flakiness (flaky tests that affect multiple projects) and inconsistent flakiness (where a test behaves non-deterministically in some projects but remains stable in others).

XII. Conclusion and Lessons Learned

Automated regression testing has become an essential component of modern code review and CI workflows. In this context, test flakiness not only disrupts individual projects, but also introduces complex challenges at the ecosystem level. In this paper, we identify and analyze cross-project flakiness and inconsistent flakiness. Below, we distill the lessons for developers, team leads, and researchers of software ecosystems.

Suggestions for Developers.

- Automate dependency validation: Our results show that inconsistent flakiness may stem from dependency issues with the patch (Observation 6). Therefore, automating dependency compatibility checks is crucial to verify that all libraries and tools across projects are version-compatible before integration (R_{12}). This proactive management of dependencies will mitigate instances of flakiness, particularly in multi-project setups.
- Adopt a preventative rather than reactive approach: More than 50% of OpenStack projects experience cross-project flakiness when rechecking failing builds (Observation 1). Non-cross-project flakiness leads to time waste, while cross-project flakiness generates even more time waste (Observation 8). This could be attributed to the "recheck and wait" paradigm prevalent in OpenStack. Therefore, we suggest that developers shift from a "recheck and wait" paradigm to early intervention in flaky tests and repeated failing builds. We encourage developers to examine even seemingly unrelated failures can prevent non-deterministic bugs reaching production, as R_8 explained "people just kept blindly reaching until they got lucky [...] to merge, [then] it becomes everyone's problem. [fixing these bugs] benefits OpenStack as a whole."
- Automate flakiness classification: We classified the root cause of inconsistent flakiness into three main categories (Observation 6), each has a unique value. We recommend extending CI tooling to classify flaky tests by their root causes (R_6 and R_7) based on the historical logs, failure symptoms, and environments, which are used to detect flaky tests in CI [2, 20, 31]. This will aid in prioritizing repair efforts effectively, e.g., event-related flakiness is likely due to external events, thus making it a low-priority task for manual log inspection and repair. For example, Brus et al. [8] propose a recheck prediction model that will automatically recheck a failing build when the predicted likelihood of passing is high. Furthermore, the root causes of cross-project and inconsistent flakiness may differ: the former can arise from intrinsic test logic [15], whereas the latter may be driven by ecosystem-level factors. A systematic classification that distinguishes between these two types of flakiness, as well as between the contexts of single-project and cross-project, could be beneficial.

Suggestions for Team Leads.

- Redesign reusable unit tests: An unexpected finding is the occurrence of flakiness in unit tests, which are traditionally designed to validate isolated components (Observations 3 and 5). This suggests a misalignment between the test design and its scope, particularly when unit tests are reused across multiple projects. Therefore, we suggest weighing the benefits of isolation testing, such as dependency injection, mocking, or stubbing, which reduce flaky dependencies on shared resources and configurations, against the potential trade-off of creating tests that are less realistic of the production environment.
- Centralize tracking of test flakiness: Cross-project flakiness often occurs in the ecosystem-level CI workflow (Observations 1 and 2), and the results of our questionnaire (R_5, R_8, and R_{15}) indicate the benefit of creating or improving a shared repository for tracking incidences of flakiness. This resource would aid in failure diagnosis and coordination of debugging effort across projects within the ecosystem. While OpenStack provides OpenSearch (i.e., a community-maintained cluster that monitors failing tests and jobs), it requires continual upkeep and active community engagement to remain viable (R_8).
- Routinely revisit CI configuration and environment: Inconsistent build configuration is a primary cause of flakiness, showing a deterministic status in other projects (Observation 6). We suggest that team leads schedule routine reviews and updates in all projects to ensure that configurations remain aligned as new practices or requirements emerge (R_{12}). Encouraging containerization or standardized testing environments across projects could mitigate environment-specific flakiness and reduce debugging time.
- Encourage multiple team collaboration and make invisible work visible: Reflecting the perspectives of R_5 and R_{10}, cross-project flakiness in the OpenStack ecosystem emerges as a socio-technical failure that necessitates coordinated effort across multiple teams. In particular, contributors (including newcomers) who experience unresponsive or delayed communication become discouraged from engaging in maintenance activities and instead adopt a "recheck and wait" paradigm. Consequently, we recommend that team leads actively foster inter-team collaboration to address cross-project flakiness and make these currently invisible maintenance efforts visible to the broader community, supported by appropriate incentives.

Suggestions for Researchers.

- Examine long-standing flakiness-related bug reports: In the open coding process of RQ3, we found that developers usually use inline comments ("recheck bug #XXXXXXX") to trigger the recheck using a persistent flakiness-related bug report. We recommend that researchers examine these bug reports (which can be

captured by terms such as "unstable," "flaky," and "non-deterministic" on Launchpad). This 2019 bug report,[11] remains unassigned, yet it is categorized by the bug supervisor as "Triaged" (indicating all necessary fix information is present) and is of medium importance.

- Analyze the long-term impact of cross-project flakiness: Cross-project flakiness generated more time waste compared to single-project flakiness (Observation 8); we suggest that researchers conduct longitudinal studies to explore its effects on long-term maintenance and software quality, such as the effectiveness of test suites over time in CI and propose strategies to mitigate these impacts.

ACKNOWLEDGMENTS

We thank all participants who took the time to complete our questionnaire, providing valuable insights for our research. We gratefully acknowledge the financial support of: (1) JSPS for the KAKENHI grants (25K22845, 23KJ1589, 26H02500, and 26K21198); (2) Japan Science and Technology Agency (JST) as part of Adopting Sustainable Partnerships for Innovative Research Ecosystem (ASPIRE), Grant Number JPMJAP2415; (3) JST PRESTO, Grant Number JPMJPR22P6; (4) the Kayamori Foundation of Informational Science Advancement for supporting Tao Xiao; (5) the Inamori Research Institute for Science for supporting Yasutaka Kamei via the InaRIS Fellowship; and (6) NSERC for supporting Shane McIntosh via Alliance International Collaboration Grant ALLRP 580835 - 22.

REFERENCES

[1] B. Adams, R. Kavanagh, A. E. Hassan, and D. M. German, "An empirical study of integration activities in distributions of open source software," *Empirical Software Engineering*, vol. 21, pp. 960–1001, 2016.

[2] A. Alshammari, P. Ammann, M. Hilton, and J. Bell, "230,439 test failures later: An empirical evaluation of flaky failure classifiers," in *2024 IEEE Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 2024, pp. 257–268.

[3] A. Alshammari, C. Morris, M. Hilton, and J. Bell, "Flakeflagger: Predicting flakiness without rerunning tests," in *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 2021, pp. 1572–1584.

[4] J. Bell, O. Legunsen, M. Hilton, L. Eloussi, T. Yung, and D. Marinov, "Deflaker: Automatically detecting flaky tests," in *Proceedings of the 40th international conference on software engineering*, 2018, pp. 433–444.

[5] J. H. Bernardo, D. A. da Costa, and U. Kulesza, "Studying the impact of adopting continuous integration on the delivery time of pull requests," in *Proceedings of the 15th International Conference on Mining Software Repositories*, 2018, pp. 131–141.

[6] C. Bogart, C. Kästner, J. Herbsleb, and F. Thung, "How to break an api: cost negotiation and community values in three software ecosystems," in *Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering*, 2016, pp. 109–120.

[7] Brown, Geoff, "Test verification," https://web.archive.org/web/20201022110253/https://developer.mozilla.org/en-us/docs/mozilla/qa/test_verification, [Accessed 17-02-2025].

[8] Y. Brus, R. Maipradit, E. T. Barr, and S. McIntosh, "Rechecking recheck requests in continuous integration," in *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2025, p. To appear.

[9] N. Cassee, B. Vasilescu, and A. Serebrenik, "The silent helper: the impact of continuous integration on code reviews," in *2020 IEEE 27th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 2020, pp. 423–434.

[10] K. Charmaz, *Constructing Grounded Theory*. SAGE, 2014.

[11] N. Cliff, "Dominance statistics: Ordinal analyses to answer ordinal questions." *Psychological bulletin*, vol. 114, no. 3, p. 494, 1993.

[12] C. R. de Souza, F. Figueira Filho, M. Miranda, R. P. Ferreira, C. Treude, and L. Singer, "The social side of software platform ecosystems," in *Proceedings of the 2016 CHI conference on human factors in computing systems*, 2016, pp. 3204–3214.

[13] A. Decan, T. Mens, and M. Claes, "An empirical comparison of dependency issues in oss packaging ecosystems," in *2017 IEEE 24th international conference on software analysis, evolution and reengineering (SANER)*. IEEE, 2017, pp. 2–12.

[14] T. Durieux, C. Le Goues, M. Hilton, and R. Abreu, "Empirical study of restarted and flaky builds on travis ci," in *Proceedings of the 17th International Conference on Mining Software Repositories*, 2020, pp. 254–264.

[15] M. Eck, F. Palomba, M. Castelluccio, and A. Bacchelli, "Understanding flaky tests: The developer's perspective," in *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2019, pp. 830–840.

[16] K. M. Eisenhardt, "Building theories from case study research," *Academy of management review*, vol. 14, no. 4, pp. 532–550, 1989.

[17] S. Fatima, T. A. Ghaleb, and L. Briand, "Flakify: A black-box, language model-based predictor for flaky tests," *IEEE Transactions on Software Engineering*, vol. 49, no. 4, pp. 1912–1927, 2022.

[18] J. L. Fleiss, "Measuring nominal scale agreement among many raters." *Psychological bulletin*, vol. 76, no. 5, p. 378, 1971.

[19] A. Foundjem and B. Adams, "Release synchronization in software ecosystems: Empirical study on openstack," *Empirical Software Engineering*, vol. 26, pp. 1–50, 2021.

[20] G. Haben, S. Habchi, J. Micco, M. Harman, M. Papadakis, M. Cordy, and Y. Le Traon, "The importance

[11] https://bugs.launchpad.net/mistral/+bug/1812171

of accounting for execution failures when predicting test flakiness," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 2024, pp. 1979–1989.

[21] K. Herzig, M. Greiler, J. Czerwonka, and B. Murphy, "The art of testing less without sacrificing quality," in *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*, vol. 1. IEEE, 2015, pp. 483–493.

[22] T. Hirao, S. McIntosh, A. Ihara, and K. Matsumoto, "The review linkage graph for code review analytics: A recovery approach and empirical study," in *Proceedings of the 2019 27th ACM joint meeting on European software engineering conference and symposium on the foundations of software engineering*, 2019, pp. 578–589.

[23] S. Jansen and M. A. Cusumano, "Defining software ecosystems: a survey of software platforms and business network governance," in *Software ecosystems*. Edward Elgar Publishing, 2013, pp. 13–28.

[24] H. Jiang, X. Li, Z. Yang, and J. Xuan, "What causes my test alarm? automatic cause analysis for test alarms in system and integration testing," in *2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE)*. IEEE, 2017, pp. 712–723.

[25] S. Khatoonabadi, D. E. Costa, R. Abdalkareem, and E. Shihab, "On wasted contributions: Understanding the dynamics of contributor-abandoned pull requests–a mixed-methods study of 10 large open-source projects," *ACM Transactions on Software Engineering and Methodology*, vol. 32, no. 1, pp. 1–39, 2023.

[26] R. Kikas, G. Gousios, M. Dumas, and D. Pfahl, "Structure and evolution of package dependency networks," in *2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR)*. IEEE, 2017, pp. 102–112.

[27] R. Kosara, F. Bendix, and H. Hauser, "Parallel sets: Interactive exploration and visual analysis of categorical data," *IEEE transactions on visualization and computer graphics*, vol. 12, no. 4, pp. 558–568, 2006.

[28] W. Lam, P. Godefroid, S. Nath, A. Santhiar, and S. Thummalapenta, "Root causing flaky tests in a large-scale industrial setting," in *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2019, pp. 101–111.

[29] W. Lam, K. Muşlu, H. Sajnani, and S. Thummalapenta, "A study on the lifecycle of flaky tests," in *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, 2020, pp. 1471–1482.

[30] W. Lam, R. Oei, A. Shi, D. Marinov, and T. Xie, "idflakies: A framework for detecting and partially classifying flaky tests," in *2019 12th ieee conference on software testing, validation and verification (icst)*. IEEE, 2019, pp. 312–322.

[31] J. Lampel, S. Just, S. Apel, and A. Zeller, "When life gives you oranges: detecting and diagnosing intermittent job failures at mozilla," in *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2021, pp. 1381–1392.

[32] F. Leinen, D. Elsner, A. Pretschner, A. Stahlbauer, M. Sailer, and E. Jürgens, "Cost of flaky tests in continuous integration: An industrial case study," in *2024 IEEE Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 2024, pp. 329–340.

[33] Q. Luo, F. Hariri, L. Eloussi, and D. Marinov, "An empirical analysis of flaky tests," in *Proceedings of the 22nd ACM SIGSOFT international symposium on foundations of software engineering*, 2014, pp. 643–653.

[34] R. Maipradit, D. Wang, P. Thongtanunam, R. G. Kula, Y. Kamei, and S. McIntosh, "Repeated builds during code review: An empirical study of the openstack community," in *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2023, pp. 153–165.

[35] K. Manikas and K. M. Hansen, "Software ecosystems–a systematic literature review," *Journal of Systems and Software*, vol. 86, no. 5, pp. 1294–1306, 2013.

[36] H. B. Mann and D. R. Whitney, "On a test of whether one of two random variables is stochastically larger than the other," *The annals of mathematical statistics*, pp. 50–60, 1947.

[37] S. McIntosh, Y. Kamei, B. Adams, and A. E. Hassan, "An empirical study of the impact of modern code review practices on software quality," *Empirical Software Engineering*, vol. 21, pp. 2146–2189, 2016.

[38] T. Mens, B. Adams, and J. Marsan, "Towards an interdisciplinary, socio-technical analysis of software ecosystem health," *arXiv preprint arXiv:1711.04532*, 2017.

[39] T. Mens and M. Goeminne, "Analysing the evolution of social aspects of open source software ecosystems." in *IWSECO@ ICSOB*, 2011, pp. 1–14.

[40] J. Micco, "The state of continuous integration testing@ google," in *ICST*, 2017.

[41] Micco, John, "Flaky tests at google and how we mitigate them," https://testing.googleblog.com/2016/05/flaky-tests-at-google-and-how-we.html, 2016, [Accessed 17-02-2025].

[42] P. Morville and L. Rosenfeld, *Information architecture for the World Wide Web: Designing large-scale web sites*. O'Reilly Media, 2006.

[43] P. Musavi, B. Adams, and F. Khomh, "Experience report: An empirical study of api failures in openstack cloud environments," in *2016 IEEE 27th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 2016, pp. 424–434.

[44] D. Neary. (2015, 3) Upstream first: Turning openstack into an nfv platform. Accessed: 2024-06-03. [Online]. Available: https://www.redhat.com/en/blog/upstream-first-turning-openstack-nfv-platform

[45] D. Olewicki, M. Nayrolles, and B. Adams, "Towards language-independent brown build detection," in *Proceedings of the 44th International Conference on Software Engineering*, 2022, pp. 2177–2188.

[46] OpenDev, "Openstack repositories hosted on opendev gerrit code review," https://review.opendev.org/admin/repos/q/filter:openstack/, [Accessed 10-09-2025].

[47] ——, "Third party testing policies for opendev,"

https://docs.opendev.org/opendev/system-config/latest/third_party.html#requirements, [Accessed 21-05-2025].
[48] OpenSearch, "Opensearch dashboards," [Accessed 2025-12-11]. [Online]. Available: https://opensearch.logs.openstack.org/_dashboards/app/login?nextUrl=%2F_dashboards%2Fapp%2Fdiscover%3Fsecurity_tenant%3Dglobal
[49] OpenStack, "2023-07-24 unmaintained status replaces extended maintenance," https://governance.openstack.org/tc/resolutions/20230724-unmaintained-branches.html, 2025, [Accessed 13-02-2025].
[50] ——, "The flaky test_servers function," https://opendev.org/openstack/openstacksdk/src/commit/d5ba0c42a9d2ed489aca3a6e431da59be3cf39c8/openstack/tests/unit/test_stats.py#L235, 2025, [Accessed 13-02-2025].
[51] ——, "How to handle test failures," https://docs.openstack.org/project-team-guide/testing.html#how-to-handle-test-failures, 2025, [Accessed 13-02-2025].
[52] ——, "The most frequent flaky api test," https://opendev.org/openstack/tempest/src/commit/357fc7ec2dfb1fb27d52cd518375e4d8ec977733/tempest/api/compute/servers/test_server_rescue.py#L230, 2025, [Accessed 13-02-2025].
[53] ——, "The most frequent flaky unit test," https://opendev.org/openstack/nova/src/commit/796250fa0d325057868572f8a4bb29aacf1edcf6/nova/tests/unit/db/main/test_migrations.py#L132, 2025, [Accessed 13-02-2025].
[54] ——, "Openstack neutron review 920766," https://review.opendev.org/c/openstack/neutron/+/920766, 2025, [Accessed 13-02-2025].
[55] ——, "Openstack neutron review 921420," https://review.opendev.org/c/openstack/neutron/+/921420, 2025, [Accessed 13-02-2025].
[56] ——, "Openstack project team guide," https://docs.openstack.org/project-team-guide/testing.html, 2025, [Accessed 13-02-2025].
[57] ——, "Testing neutron," https://docs.openstack.org/neutron/latest/contributor/testing/testing.html#structure-of-the-unit-test-tree, 2025, [Accessed 10-12-2025].
[58] OpenStack Neutron, "Full stack testing — neutron 28.0.0.0b2.dev20 documentation," [Accessed 12-11-2025]. [Online]. Available: https://docs.openstack.org/neutron/latest/contributor/testing/fullstack.html
[59] O. Parry, G. M. Kapfhammer, M. Hilton, and P. McMinn, "A survey of flaky tests," *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 31, no. 1, pp. 1–74, 2021.
[60] M. M. Rahman and C. K. Roy, "Impact of continuous integration on code reviews," in *2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR)*. IEEE, 2017, pp. 499–502.
[61] Raine, Jordan, "Github: Reducing flaky builds by 18x," https://github.blog/engineering/engineering-principles/reducing-flaky-builds-by-18x/, 2020, [Accessed 26-02-2025].
[62] Red Hat Bugzilla, "Bug 1833757," https://bugzilla.redhat.com/show_bug.cgi?id=1833757, 2025, [Accessed 13-02-2025].
[63] M. Riedemann. (2018, 4) Seeding the future of extended maintenance in openstack. Accessed: 2024-06-03. [Online]. Available: https://superuser.openinfra.dev/articles/extended-maintenance-openstack/
[64] P. C. Rigby and C. Bird, "Convergent contemporary software peer review practices," in *Proceedings of the 2013 9th joint meeting on foundations of software engineering*, 2013, pp. 202–212.
[65] P. C. Rigby and M.-A. Storey, "Understanding broadcast based peer review on open source software projects," in *Proceedings of the 33rd International conference on software engineering*, 2011, pp. 541–550.
[66] J. Romano, J. D. Kromrey, J. Coraggio, J. Skowronek, and L. Devine, "Exploring methods for evaluating group differences on the NSSE and other surveys: Are the t-test and Cohen's d indices the most appropriate choices?" in *Annual Meeting of the Southern Association for Institutional Research*, 2006, pp. 1–51.
[67] stestr, "stestr user manual," [Accessed 2025-12-11]. [Online]. Available: https://stestr.readthedocs.io/en/latest/MANUAL.html#configuration
[68] unittest, "unittest — unit testing framework," [Accessed 2025-12-11]. [Online]. Available: https://docs.python.org/3/library/unittest.html#test-discovery
[69] A. Vahabzadeh, A. M. Fard, and A. Mesbah, "An empirical study of bugs in test code," in *2015 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, 2015, pp. 101–110.
[70] ——, "An empirical study of bugs in test code," in *2015 IEEE international conference on software maintenance and evolution (ICSME)*. IEEE, 2015, pp. 101–110.
[71] A. J. Viera, J. M. Garrett *et al.*, "Understanding interobserver agreement: the kappa statistic," *Fam med*, vol. 37, no. 5, pp. 360–363, 2005.
[72] D. Wang, Y. Ueda, R. G. Kula, T. Ishio, and K. Matsumoto, "Can we benchmark code review studies? a systematic mapping study of methodology, dataset, and metric," *Journal of Systems and Software*, vol. 180, p. 111009, 2021.
[73] D. Wang, T. Xiao, P. Thongtanunam, R. G. Kula, and K. Matsumoto, "Understanding shared links and their intentions to meet information needs in modern code review: A case study of the openstack and qt projects," *Empirical Software Engineering*, vol. 26, pp. 1–32, 2021.
[74] T. Xiao, D. Wang, S. McIntosh, H. Hata, R. G. Kula, T. Ishio, and K. Matsumoto, "Characterizing and mitigating self-admitted technical debt in build systems," *IEEE Transactions on Software Engineering*, vol. 48, no. 10, pp. 4214–4228, 2021.
[75] T. Xiao, D. Wang, S. McIntosh, Y. Kamei, and H. Hata, "Research artifact – cross-project flakiness: A case study of the openstack ecosystem," https://doi.org/10.5281/zenodo.18476912, 2026, [Accessed 04-02-2026].
[76] R. K. Yin, *Case study research: Design and methods*.

sage, 2009, vol. 5.

[77] F. Zampetti, G. Bavota, G. Canfora, and M. Di Penta, "A study on the interplay between pull request review and continuous integration builds," in *2019 IEEE 26th international conference on software analysis, evolution and reengineering (SANER)*. IEEE, 2019, pp. 38–48.

[78] S. Zhang, D. Jalali, J. Wuttke, K. Muşlu, W. Lam, M. D. Ernst, and D. Notkin, "Empirically revisiting the test independence assumption," in *Proceedings of the 2014 International Symposium on Software Testing and Analysis*, 2014, p. 385–396.

[79] Zuul, "Nodepool v3.8.0 configuration," https://zuul-ci.org/docs/nodepool/3.8.0/configuration.html#attr-build-log-retention, [Accessed 21-05-2025].

[80] ——, "Projects hosted on zuul ci platform," https://zuul.opendev.org/t/openstack/projects, [Accessed 11-03-2025].